

Final Project: Comprehensive Analysis and Classification of Wheat Seed Dataset Using Various Machine Learning Techniques

Yixiang Xu 12313422

Abstract:

This study investigates the effectiveness of several machine learning techniques on the Wheat Seed dataset, focusing on both multi-class and binary classification. Methods include K-Means clustering (with K-Means++ initialization and Soft K-Means), Principal Component Analysis (PCA), Nonlinear Autoencoders, and supervised learning techniques such as Multi-Layer Perceptron (MLP), Support Vector Machines (SVM), and AdaBoost. For binary classification, results highlight the comparative performance of MLP, linear SVM, Gaussian kernel SVM, and AdaBoost. The evaluation metrics include accuracy, precision, recall, F1-score, and confusion matrices, providing insights into the advantages and limitations of each approach.

Contents:

1. Introduction

2. Problem Description

3. Methodology

3.0 Preprocessing of data

3.1 K-Means++ Clustering

3.2 Soft K-Means Clustering

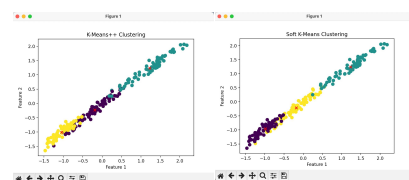
3.3 Dimensionality Reduction (PCA and Nonlinear Autoencoders)

3.4 Supervised Learning for Multi-Class Classification

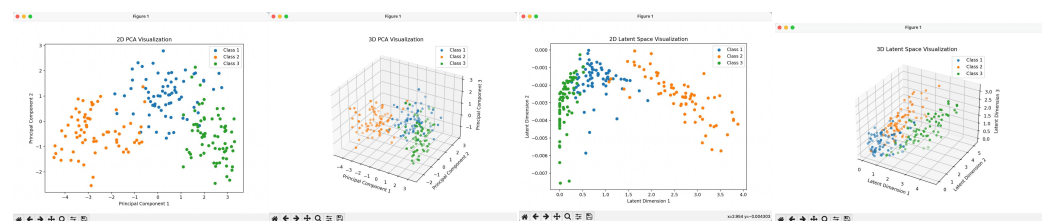
3.5 Supervised Learning for Binary Classification

4. Experiments and Results

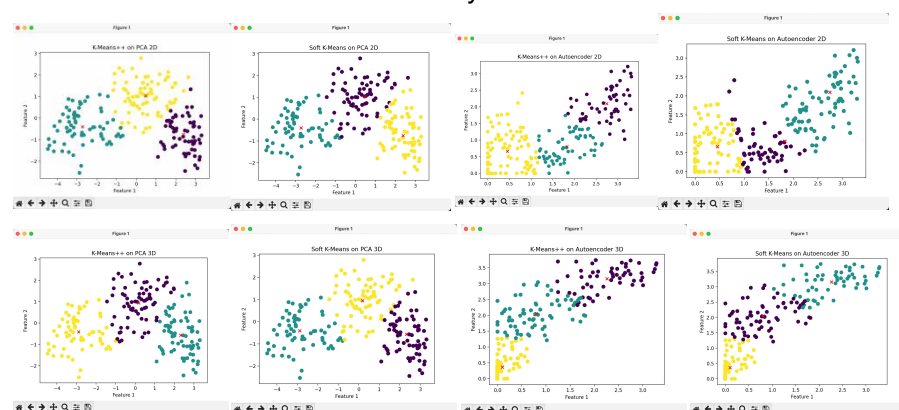
4.1 Clustering Results



4.2 Dimensionality Reduction (PCA and Autoencoder)



4.3 Multi-Class Classification Analysis with confusion matrix



4.4 Binary Classification Analysis

5. Some other discussion

6. Conclusion

1. Instruction

The Wheat Seed dataset provides three classes of wheat seeds, characterized by specific features. This report addresses the challenges of clustering, reducing dimensionality, and classifying data accurately using modern machine learning techniques. Emphasis is placed on comparing the efficacy of these techniques to offer insights into their application in similar domains. In order to simplify my coder and improve the readability of it, I choose to use “class” to packaging the methods of each multiple-classification. Using the former code helps a lot.

2. Problem Description

The Wheat Seed dataset comprises samples labeled into three classes, each with seven features such as area, perimeter, and compactness. The main objectives are:

(1)To effectively cluster and classify the dataset using unsupervised and supervised learning techniques.

(2)To evaluate binary classification performance by separating one class against another.

(3)Summarize the findings and provide conclusions based on the performance of the different machine learning techniques.

3. Methodology

3.0 Preprocessing of data:

First of all steps, I should reference some libraries and do preprocess of data, including loading data, the normalization, slipping the dataset with random and define some functions we use in the latter steps. (for my project is sigmoid, relu leaky_relu and their derivatives)

3.1K-Means++ Clustering:

I define three function in class “Kpp” meas K++, utilizing the K-Means++ initialization to enhance clustering stability, visualizing clusters in reduced dimensions. The first centroid is randomly selected, and then the subsequent centroids are chosen based on the distance between the existing centroids and the data points, according to a probability distribution based on the distances. It is noticeably that I use “tol=1e-4” to judge if it is good enough to output the centroids of different labels. It means that if the change of the two centroids of two following trains is lower than tol, the trian step can stop, and I think it is good enough.

3.2 Soft K-Means Clustering:

Almost as same as K-means++, just convert the hard labels to the possibilities, which symbolizes “soft”.

3.3 Dimensionality Reduction (PCA and Nonlinear Autoencoders):

(1) PCA:

The key point is the covariance matrix, which reflects the linear relationship between the individual features, then solve the eigenvalues and eigenvectors of the covariance matrix. The last step is sorting the eigenvalues by size, and select the corresponding eigenvectors (principal components) in the order in which they are sorted, then, take the first `n_components` principal components. I use scatter plot to do visualization.

(2) Nonlinear Autoencoders:

I choose `leaky_relu` and its derivative as the activation part, which is defined at the first of the project. By using a nonlinear activation function, it can capture the more complex nonlinear features of the input data. It allows some non-zero gradients in the negative input section, which solves the problem of "dead neurons" in ReLU. In this way, to a certain extent, it can avoid overfitting, increase the speed of model convergence, make radiant vanishing occurs less often.

(3) Clustering with Reduced Dimensions:

In this part, it is required to apply K-Means++ and soft K-means on the reduced dimensions obtained from PCA and nonlinear autoencoder, respectively. I realize it in both 2D and 3D. In order to evaluate the representation of the different method, we use accuracy, precision, recall, F1 score to do it. However, it is necessary to ensure that the clustering results can be compared with the real labels. The Hungarian algorithm was used to align the category assignments of the clustering results with the real labels.

```
#Cluster alignment
def map_cluster_to_labels(self, labels, y_true):
    """
    Map clustering labels to ground truth labels using maximum assignment.

    Parameters:
        labels (numpy.ndarray): Clustering labels of shape (n_samples,).
        y_true (numpy.ndarray): True labels of shape (n_samples,).

    Returns:
        mapped_labels (numpy.ndarray): Remapped clustering labels aligned with true labels.
    """
    unique_clusters = np.unique(labels)
    unique_classes = np.unique(y_true)

    # Create a cost matrix where the value is the count of matching pairs
    cost_matrix = np.zeros((len(unique_clusters), len(unique_classes)))

    for i, cluster in enumerate(unique_clusters):
        for j, true_class in enumerate(unique_classes):
            cost_matrix[i, j] = np.sum((labels == cluster) & (y_true == true_class))

    # Compute mapping by maximizing assignment
    mapping = {}
    for cluster_idx in range(len(unique_clusters)):
        max_class_idx = np.argmax(cost_matrix[cluster_idx])
        mapping[unique_clusters[cluster_idx]] = unique_classes[max_class_idx]

    # Map labels using the computed mapping
    mapped_labels = np.array([mapping[label] for label in labels])
    return mapped_labels
```

3.4 Supervised Learning for Multi-Class Classification:

(1) MLP for Multi-Class Classification:

The first step in MLP is converting tags to one-hot encoding. Here I choose sigmoid and softmax as activation functions, in order to fit the multiple classification. As for loss, I choose cross-entropy as the loss function to measure the difference between the predicted probability distribution and the true label distribution. Then comes the forward and backward propagation and the update of weight and bias, which are all called by the

train function. (The comparison of MLP and clustering has been shown in last class.)

(2)SVM and SVM with Gaussian Kernel:

The core idea is to break down a multi-class problem into multiple dichotomous problems, and then distinguish each category from the others. I choose 1-vs-rest method to realize multiple classification. The RBF kernel to compute the similarity in the feature space allows the model to handle cases where it is nonlinearly separable. Unlike gradient updates in linear SVMs, the training process here involves calculating the Gaussian kernel matrix K and updating the α coefficients. Due to the use of nonlinear kernels, the decision boundaries of the classifier can be more complex and suitable for nonlinearly separable data. Key differences: Kernel Functions: The previous implementation was a linear SVM, while the Gaussian Radial Kernel (RBF Kernel) is used here to handle non-linear separability. The RBF kernel enables support vector machines to map the input space to a high-dimensional space, looking for more complex decision boundaries in the high-dimensional space. Decision function: The decision function of a linear SVM is based on the linear inner product, while the decision function of a Gaussian kernel SVM is based on the similarity calculated by the kernel function.

(3)AdaBoost Algorithm:

AdaBoost forms a strong classifier by combining multiple weak classifiers. The weight of each weak classifier is calculated based on its error rate, and the misclassified samples are given a higher weight, so that subsequent weak classifiers will pay more attention to the samples that are difficult to classify. This implementation uses a one-to-many (One-vs-Rest) strategy to transform a multi-class problem into multiple binary classification problems, training an AdaBoost classifier for each class. When forecasting, the final category is selected based on the score of each category. For each class, treat it as a positive class and the others as negatives, and train a dichotomous AdaBoost model.

3.5 Supervised Learning for Binary Classification:

I first remove label 2 and convert the other labels to 1 and -1 for binary classification. Then implement MLP, SVM, SVM with Gaussian, and AdaBoost step by step.

(1)MLP for binary classification: As the dataset is not very complex, I just use batch gradient descent rather than mini-batch gradient descent, and choose sigmoid and relu as activation functions. Different with other binary classifications, MLP needs 1 and 0 labels rather than 1 and -1, so I convert the label for MLP. Others are nearly as same as multiple classifications.

(2)The evaluations of binary classifications: I use “!=1” symbolize 0 and -1. Only in this way can I unify the evaluation of MLP and other methods.

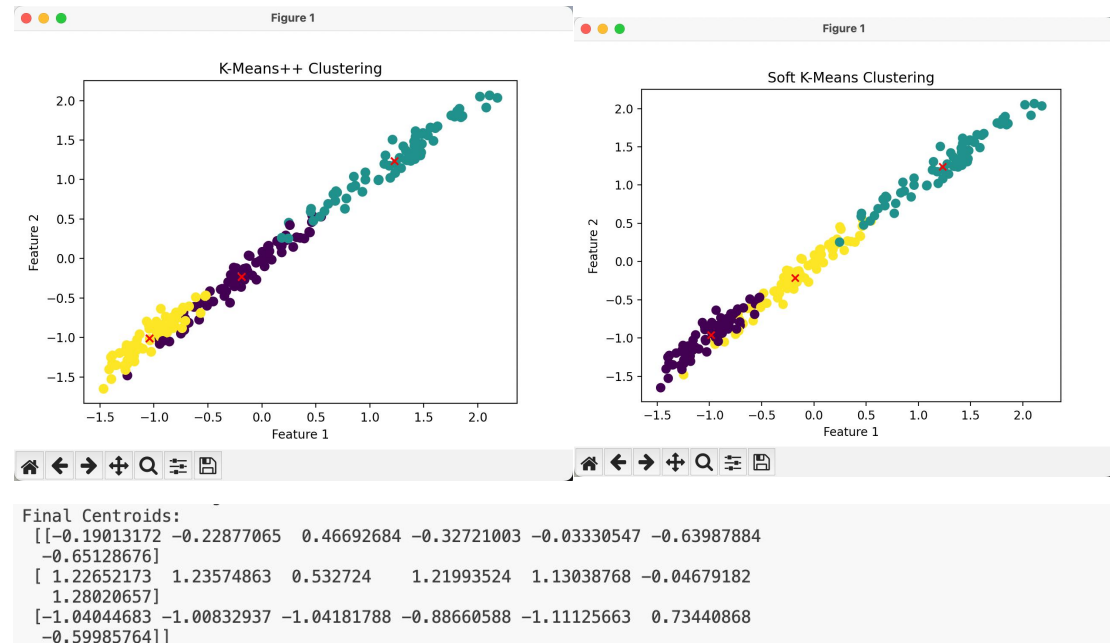
```
# Calculate confusion matrix elements
true_positive = np.sum((y_true == 1) & (y_pred == 1)) # Correctly predicted positive class
true_negative = np.sum((y_true != 1) & (y_pred != 1)) # Correctly predicted negative class
false_positive = np.sum((y_true != 1) & (y_pred == 1)) # Negative class incorrectly predicted as positive
false_negative = np.sum((y_true == 1) & (y_pred != 1)) # Positive class incorrectly predicted as negative

# Calculate accuracy, precision, recall, and F1 score
# Precision = TP / (TP + FP)
# Recall = TP / (TP + FN)
# F1 score = 2 * (precision * recall) / (precision + recall)
accuracy = (true_positive + true_negative) / (true_positive + true_negative + false_positive + false_negative)
precision = true_positive / (true_positive + false_positive) if (true_positive + false_positive) > 0 else 0
recall = true_positive / (true_positive + false_negative) if (true_positive + false_negative) > 0 else 0
f1 = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0
```

4. Experiments and Results

4.1 Clustering Results:

(1):K-Means++: Achieved compact clusters visualized in 2D using PCA. The algorithm converged in fewer iterations, ensuring stable centroid placement.



(2):Soft K-Means: Enhanced interpretation through probabilistic soft assignments but required careful parameter tuning.

Soft K-Means Clustering:

Final Centroids:

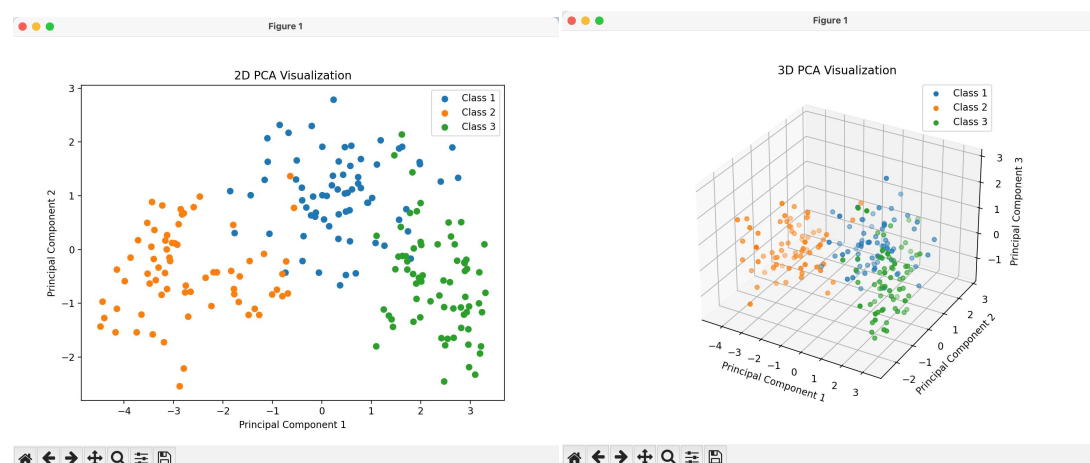
```
[[-0.98465231 -0.95913555 -0.92591114 -0.8522671 -1.04247201 0.65986174  
-0.61236009]  
[ 1.23232902 1.23785508 0.55295587 1.22235158 1.14044509 -0.0798563  
1.25777046]  
[-0.18012254 -0.21089082 0.40372651 -0.30319525 -0.03533136 -0.58494837  
-0.57683393]]
```

First 5 Soft Assignments:

```
[[0.01827581 0.02680653 0.95491765]  
[0.01380406 0.00866736 0.97752858]  
[0.02270628 0.00405778 0.97323594]  
[0.02159915 0.00262792 0.97577293]  
[0.0071577 0.09774408 0.89509821]]
```

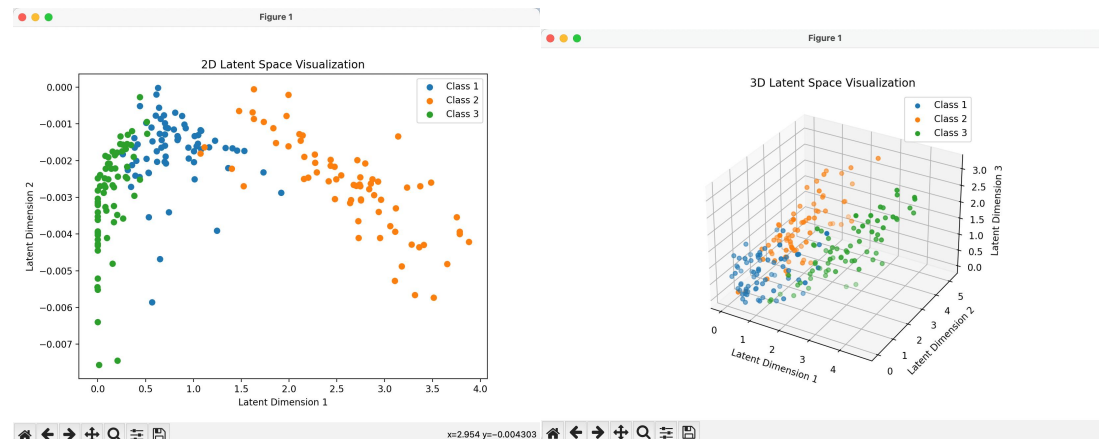
4.2 Dimensionality Reduction (PCA and Autoencoder)

(1) PCA: Top 2 and 3 components visualized, capturing major variance.

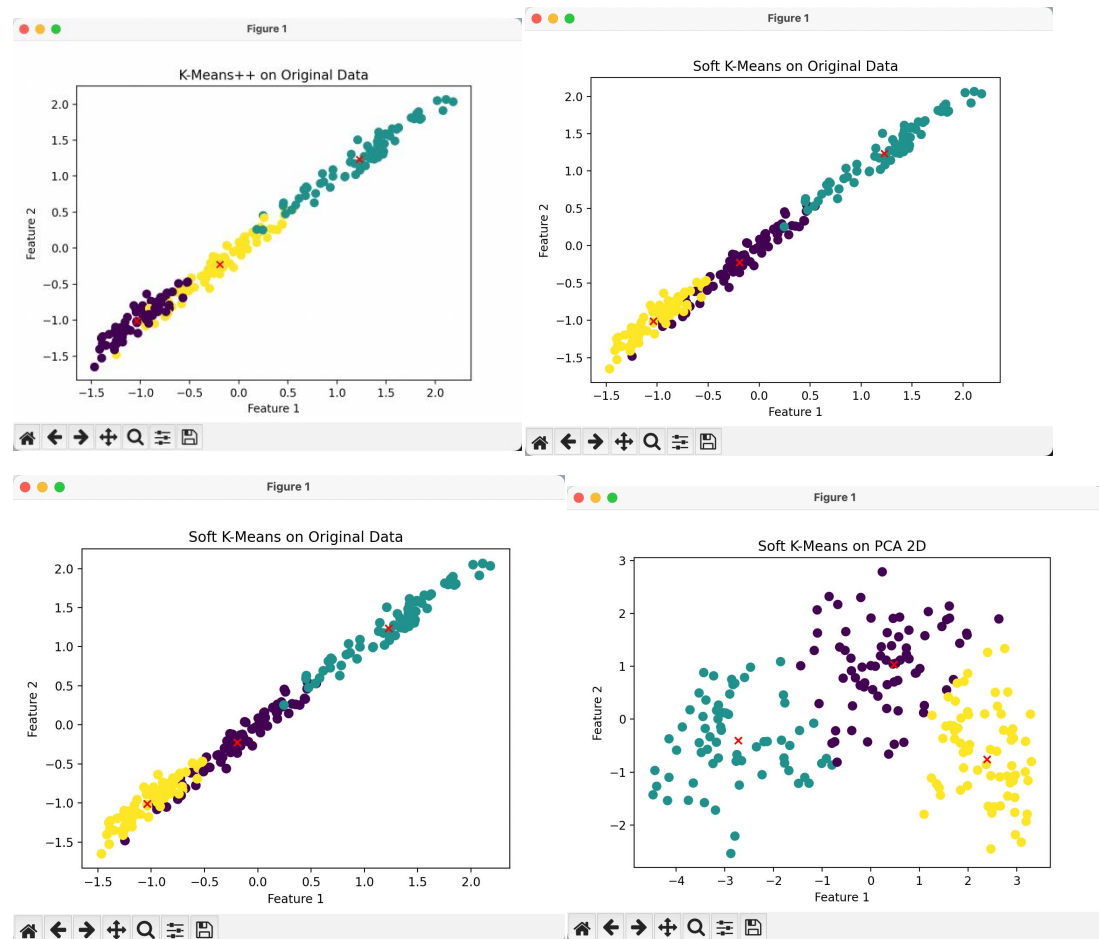


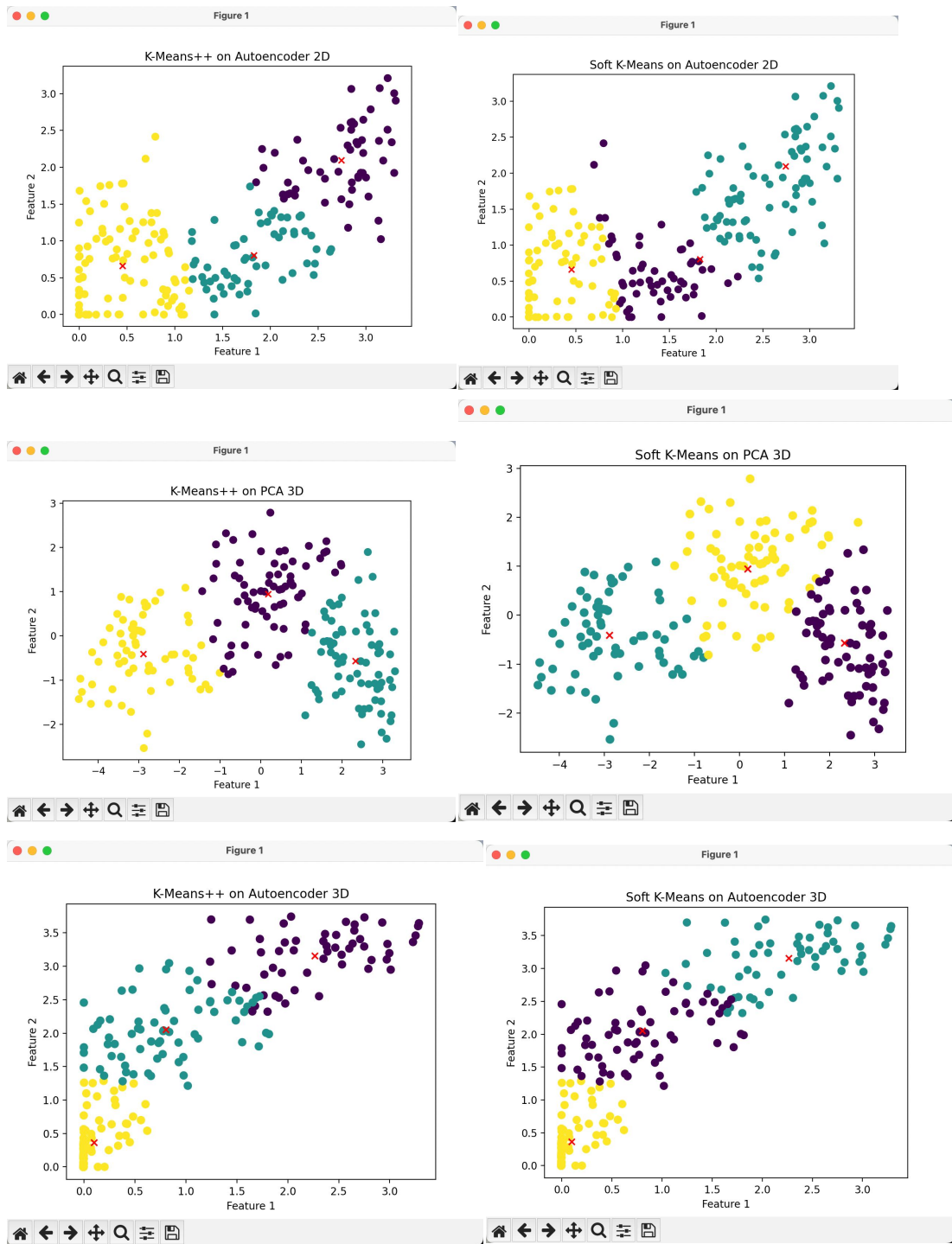
From the pictures, it has a good performance in the dataset.

(2): Autoencoder: Reduced dimensions while preserving nonlinear relationships.



(3)Clustering with Reduced Dimensions:





From the results, it is clear that MLP for multiple classification has a better performance than reduced dimensional clustering.

```
Clustering on Original Data:
K-Means++ Metrics on Original Data: {'Accuracy': 0.9380952380952381, 'Precision': 0.9391026015316255, 'Recall': 0.9380952380938979, 'F1-score': 0.9385986494713}
Soft K-Means Metrics on Original Data: {'Accuracy': 0.9238095238095239, 'Precision': 0.9249089061720343, 'Recall': 0.9238095238082039, 'F1-score': 0.9243588880537847}
```

```
Clustering on PCA Reduced Data (2D):
K-Means++ Metrics on PCA 2D: {'Accuracy': 0.919047619047619, 'Precision': 0.9214407814394572, 'Recall': 0.9190476190463062, 'F1-score': 0.9202426442943935}
Soft K-Means Metrics on PCA 2D: {'Accuracy': 0.9238095238095239, 'Precision': 0.9249089061720343, 'Recall': 0.9238095238082039, 'F1-score': 0.9243588880537847}
```

```
Clustering on Autoencoder Reduced Data (2D):
Epoch 10/50, Loss: 0.356698
Epoch 20/50, Loss: 0.158078
Epoch 30/50, Loss: 0.160466
Epoch 40/50, Loss: 0.129257
Epoch 50/50, Loss: 0.124909
K-Means++ Metrics on Autoencoder 2D: {'Accuracy': 0.7761904761904762, 'Precision': 0.7898454887849479, 'Recall': 0.7761904761893673, 'F1-score': 0.7829584501616723}
Soft K-Means Metrics on Autoencoder 2D: {'Accuracy': 0.8380952380952381, 'Precision': 0.8362849105874886, 'Recall': 0.8380952380940409, 'F1-score': 0.8371890956343695}
```

```
Clustering on PCA Reduced Data (3D):
K-Means++ Metrics on PCA 3D: {'Accuracy': 0.8952380952380953, 'Precision': 0.8979917417404497, 'Recall': 0.8952380952368163, 'F1-score': 0.8966128042174356}
Soft K-Means Metrics on PCA 3D: {'Accuracy': 0.9238095238095239, 'Precision': 0.9249089061720343, 'Recall': 0.9238095238082039, 'F1-score': 0.9243588880537847}
```

```
Clustering on Autoencoder Reduced Data (3D):
Epoch 10/50, Loss: 0.376593
Epoch 20/50, Loss: 0.201796
Epoch 30/50, Loss: 0.145719
Epoch 40/50, Loss: 0.103292
Epoch 50/50, Loss: 0.068358
K-Means++ Metrics on Autoencoder 3D: {'Accuracy': 0.7904761904761904, 'Precision': 0.7920634920623245, 'Recall': 0.7904761904750611, 'F1-score': 0.7912690451797825}
Soft K-Means Metrics on Autoencoder 3D: {'Accuracy': 0.8, 'Precision': 0.8004282172627449, 'Recall': 0.79999999999988571, 'F1-score': 0.8002140512930005}
```

MLP Accuracy: 95.24%

```
Comparing MLP with Clustering Results:
MLP Accuracy: 0.9524
Comparing MLP with K-Means++ and Soft K-Means on Original Data:
K-Means++ Accuracy: 0.9381
Soft K-Means Accuracy: 0.9238
```

```
Comparing MLP with K-Means++ and Soft K-Means on PCA 2D Data:
K-Means++ Accuracy on PCA 2D: 0.8952
Soft K-Means Accuracy on PCA 2D: 0.9238
```

```
Comparing MLP with K-Means++ and Soft K-Means on Autoencoder 2D Data:
K-Means++ Accuracy on Autoencoder 2D: 0.7905
Soft K-Means Accuracy on Autoencoder 2D: 0.8000
```

```
Comparing MLP with K-Means++ and Soft K-Means on PCA 3D Data:
K-Means++ Accuracy on PCA 3D: 0.8952
Soft K-Means Accuracy on PCA 3D: 0.9238
```

```
Comparing MLP with K-Means++ and Soft K-Means on Autoencoder 3D Data:
K-Means++ Accuracy on Autoencoder 3D: 0.7905
Soft K-Means Accuracy on Autoencoder 3D: 0.8000
```

4.3 Multi-Class Classification Analysis, with confusion matrix

(1):MLP for Multi-Class Classification:Achieved an accuracy of 95.24% on the test set with improved generalization using a tanh activation and cross-entropy loss.

(2)SVM and SVM with Gaussian Kernel: Simpler decision boundaries, slightly lower accuracy than MLP due to linearly separable assumption.

Linear SVM: 92.86% accuracy

Gaussian Kernel SVM: 92.86% accuracy

```

Training Linear SVM for Multi-class...
Accuracy: 92.86%
Confusion Matrix:
[[ 9  1  2]
 [ 0 15  0]
 [ 0  0 15]]
Training Gaussian Kernel SVM for Multi-class...
Accuracy: 92.86%
Confusion Matrix:
[[10  1  1]
 [ 0 15  0]
 [ 1  0 14]]

```

(3)AdaBoost Algorithm: Demonstrated robustness against noise, achieving a balance between bias and variance, with accuracy of 95.24%.

```

Training AdaBoost for Multi-class...
AdaBoost Multi-class Accuracy: 95.24%
Confusion Matrix:
[[11.  1.  0.]
 [ 0. 15.  0.]
 [ 1.  0. 14.]]

```

4.4 Binary Classification Analysis

(1)MLP: Accuracy 92.59%, with a good precision of 100%.

```

Training MLP...
Epoch 100/1000, Loss: 0.864914
Epoch 200/1000, Loss: 0.825325
Epoch 300/1000, Loss: 0.806347
Epoch 400/1000, Loss: 0.803562
Epoch 500/1000, Loss: 0.812287
Epoch 600/1000, Loss: 0.828490
Epoch 700/1000, Loss: 0.849245
Epoch 800/1000, Loss: 0.872496
Epoch 900/1000, Loss: 0.896929
Epoch 1000/1000, Loss: 0.921733
Accuracy: 92.59%
Precision: 100.00%
Recall: 83.33%
F1-Score: 90.91%
True Positive: 10
True Negative: 15
False Positive: 0
False Negative: 2

```

(2)Linear SVM: Accuracy 96.30%. Effective with linear separability, but performance degraded with high-dimensional data.

```

Training Linear SVM...
Accuracy: 96.30%
Precision: 100.00%
Recall: 91.67%
F1-Score: 95.65%
True Positive: 11
True Negative: 15
False Positive: 0
False Negative: 1

```

(3)Gaussian SVM: Accuracy 92.59%. Captured non-linear boundaries effectively, though computationally intensive.

```

Training Gaussian Kernel SVM...
Accuracy: 92.59%
Precision: 100.00%
Recall: 83.33%
F1-Score: 90.91%
True Positive: 10
True Negative: 15
False Positive: 0
False Negative: 2

```

(4)AdaBoost: Accuracy 92.59%. Resilient against outliers but sensitive to noisy features.

```

Training AdaBoost...
Accuracy: 92.59%
Precision: 85.71%
Recall: 100.00%
F1-Score: 92.31%
True Positive: 12
True Negative: 13
False Positive: 2
False Negative: 0

```

a. From the results, all the models have a good precision of 100% except AdaBoost, so it is not suitable for high-risk decision-making tasks such as medical diagnosis, financial fraud detection, legal compliance. In the wheat seed dataset, it is not suitable

for avoiding falsely classifying seeds into the wrong type (i.e., minimizing false positives), which could result in incorrect categorization of wheat seed types.

b. AdaBoost has the best recall, better than linear SVM, better than MLP and Gaussian SVM, so it is suitable for high-risk tasks with low tolerance for false negatives (misses), especially in scenarios related to personal safety, property security, health, and crime prevention. In the wheat seed dataset, it has the best performance of ensuring that as many wheat seeds as possible are correctly identified, minimizing false negatives (misidentification of wheat seed types).

c. The accuracy and F1 score of these models are near and all bigger than 90%, symbolizing the comprehensive performances are all good enough.

5. Some other discussion

5.1 whether need split train and test

Method	Data Loading and Preprocessing	Requires Train/Test Split
Data Loading and Preprocessing	Standardize data (mean=0, std=1); convert labels to 1 and -1	Yes
K-Means++	Standardize data; perform clustering directly	No
PCA	Standardize data; extract principal components for dimensionality reduction and visualization	No
Non-linear Autoencoder	Standardize data; train the model to generate low-dimensional representations	Yes
MLP (Multi-class/Binary)	Standardize data; convert labels to one-hot (multi-class) or binary (binary classification)	Yes
SVM	Standardize data; keep labels as 1 and -1	Yes
AdaBoost	Standardize data; keep labels as 1 and -1	Yes

5.2 Summary of Label Use:

(1) Unsupervised Methods (K-Means++, PCA, Autoencoder): Labels are not needed, as these methods focus on clustering or feature transformations.

(2) Supervised Methods (MLP, SVM, AdaBoost): Labels are required and must be in binary form (1 and -1).

Methods and Label Formats

Method	Label Format	Description
K-Means++	N/A	K-Means++ is an unsupervised method, so it does not use labels.
PCA	N/A	PCA is a dimensionality reduction technique, so labels are not required.
Non-linear Autoencoder	N/A	Autoencoders are unsupervised; they do not use labels but focus on reconstructing the input features.
MLP (Binary Classification)	1 and -1	Labels are binary, formatted as 1 for the positive class and -1 for the negative class. These are directly used during training and evaluation.
SVM (Binary Classification)	1 and -1	Similar to MLP, labels are binary (1 and -1), as required for hinge-loss-based SVM optimization.
AdaBoost (Binary Classification)	1 and -1	Labels are binary (1 and -1), as AdaBoost uses weighted voting for classification.

6.Conclusion

The study highlights that no single technique universally outperforms others across all tasks.

(1)Clustering: K-Means++ is computationally efficient and interpretable, while Soft K-Means offers flexibility.

(2)Dimensionality Reduction: Nonlinear Autoencoders better preserve intricate data patterns than PCA.

(3)Multi-Class Classification: MLP and AdaBoost provide robust solutions, while Linear SVMs excel in simplicity.

(4)Binary Classification: Gaussian Kernel SVMs offer superior non-linear decision boundaries, while AdaBoost provides ensemble robustness.

Future work includes exploring hyperparameter optimization and additional ensemble methods to further enhance model performance.